

--	--	--	--	--	--	--	--	--	--

Date: 09/04/2026

Duration: 3 hours

FINAL ASSESSMENT

II YEAR – SEMESTER 4

CSE23AE204 - PCP III

(B.Tech. E01,E02,E03,E05,E06)

SET B

Question 1: Problem Statement

You are developing a **premium calculation module** for an insurance company.

The system processes **one insurance policy at a time**, provided as a **space-separated string**.

Each policy belongs to a specific category and follows different premium calculation rules.

The system must:

- Identify the policy type from input
- Calculate the final premium amount
- Use **abstraction and polymorphism** to support multiple policy types
- Assign a unique policy ID using a **static variable**
- Return the calculated premium

Premium Rules

Health Insurance Policy

- A base premium is charged.
- An age-based risk amount is added.
- The total premium is the sum of base premium and risk amount.

Vehicle Insurance Policy

- A base premium is charged.
- A percentage-based vehicle value charge is added.
- The total premium is the sum of base premium and value-based charge.

Life Insurance Policy

- A base premium is charged.
- A fixed coverage amount is added.
- The total premium is the sum of base premium and coverage amount.

Example 1

Input

{ "policy": "HEALTH Ravi 5000 1500" }

Output

6500.0

Explanation

The health policy premium is calculated by adding the age-based risk amount to the base premium.

Question 2: Problem Statement

You are given a string **s** consisting of uppercase English letters.

Your task is to compress the string using a **custom adaptive run-length encoding rule** based on the **frequency comparison with the previous group**.

Encoding Rules

1. Group **consecutive identical characters**.
2. Let: currFreq = frequency of the current character group and prevFreq = frequency of the previous character group
3. Encoding rule:
 - **First group** → encode as <character><frequency>
 - For every next group: If currFreq < prevFreq → encode as <character><frequency>

Example 1

Input

ABBBCCDAAAE

Output

A3BC2D3AE2

- Else: → encode as <frequency><character>

4. If frequency is 1, **omit the number**. and 5. Maintain the original order of characters.

Question 3: Problem Statement

You are given a JSON object:

```
{ "g": [g1, g2, g3, ...], "s": [s1, s2, s3, ...] }
```

Where:

- $g[i]$ represents the **greed factor** of the i th child.
- $s[j]$ represents the **size of the j th cookie**.

A child can be satisfied if:

$\text{cookie_size} \geq \text{greed_factor}$

Rules:

- Each child can receive **at most one cookie**.
- Each cookie can be assigned to **only one child**.

Your task is to **maximize the number of satisfied children**.

Question 4: Problem Statement

You are given an **unsorted array of integers** `nums` of size n .

Your task is to determine the **maximum difference between two consecutive elements after the array is sorted in non-decreasing order**.

Steps involved:

1. Sort the array in ascending order.
2. Examine each pair of **adjacent elements** in the sorted array.
3. Compute the difference between those elements.
4. Return the **maximum difference** found among those pairs.

If the array contains **less than two elements**, return **0**, since there are no pairs to compare.

Question 5: Problem Statement

You are given a triangle array `triangle`, where:

- `triangle[i]` represents the i -th row
- Each row has one more element than the previous

You start at the **top of the triangle** and move to the bottom.

At each step, you may move to:

- The same index Or the next index $j+1$ in the next row

Your task is to find the **minimum path sum** from top to bottom.

Example 1

Input: { "g": [1,2,3], "s": [1,1] }

Output: 1

Explanation

Children greed factors: [1,2,3]

Cookies: [1,1]

Only one child with greed 1 can be satisfied.

Result: 1

Example 1

Input: `nums = [3,6,9,1]`

Output: 3

Explanation

Sorted array: [1,3,6,9]

Differences: $3 - 1 = 2$, $6 - 3 = 3$, $9 - 6 = 3$

Maximum difference = 3

Example 1:

Input:

`triangle = [ [2], [3,4], [6,5,7], [4,1,8,3] ]`

Output: 11

Explanation:

Path: $2 \rightarrow 3 \rightarrow 5 \rightarrow 1 = 11$